



Programmiersvorkurs

Einführung in Java – Tag 2

Ablauf



Montag – Donnerstag

Zeitraum	Was?
10 – 12 Uhr	Informationen, Übungsbesprechung, Vorlesung
12 – 13 Uhr	Mittagspause
ab 13 Uhr	Übungen, gemütliches Zusammensitzen

Inhaltsübersicht



Tag 1	Variablen, Datentypen, Variablennamen, Verwendung, Arithmetik, Konvertierung, Kommentare, Technologie, Eclipse Livedemo
Tag 2	Boolsche Ausdrücke, Kommentare, If-Abfragen, Switch-Case
Tag 3	Arrays, (Do-)While-Schleife, For-Schleifen, Weiterführung Debugging
Tag 4	(statische) Methoden, Klassenvariablen, JavaDoc, Exceptions

Kommentare



- Erklärt, was der Code macht, besonders für komplexe Logik.
- Hilft anderen Entwicklern, den Code leichter zu verstehen.
- Mit Javadoc-Kommentaren können automatisch HTML-Dokumentationen generiert werden.
- Schnelles Testen und Deaktivieren von Code für Debugging
- Haben **keinen** Einfluss auf den Programmablauf, wird vom Compiler ignoriert

Best Practices bei Kommentaren + IKA

- "So knapp wie möglich, so ausführlich wie nötig!„
- TODOs einbauen als Gedächtnisstütze
- Nicht offensichtlichen Code erklären.
- Vermeide übermäßige Kommentare – der Code sollte sich selbst erklären, wo möglich.
- In einer Sprache bleiben

Arten von Kommentaren



Einzeilige Kommentare

```
01 //Dies ist ein Kommentar (Endet mit Zeilenumbruch)
```

bsp.

```
01 //TODO: Feature einfügen
```

Arten von Kommentaren



Blockkommentar

```
01  /*
02  * Hiermit kann man mehrzeilige Kommentare
03  * schreiben und viele Infos
04  * unterbringen
05  *
06  */
```

Arten von Kommentaren



Javadoc

```
01  /**
02  * Kommentar, der automatisch zu HTML-Dokumentation
03  * verarbeitet wird
04  */
```


Verwendung der Methode mit JavaDoc



```
long rounded = Math.round(i);
```

© java.lang.Math

@Contract(pure = true) ↗ ↗

```
public static long round(
    double a
)
```

Returns the closest `long` to the argument, with ties rounding to positive infinity.

Special cases:

- If the argument is NaN, the result is 0.
- If the argument is negative infinity or any value less than or equal to the value of `Long.MIN_VALUE`, the result is equal to the value of `Long.MIN_VALUE`.
- If the argument is positive infinity or any value greater than or equal to the value of `Long.MAX_VALUE`, the result is equal to the value of `Long.MAX_VALUE`.

Params: `a` – a floating-point value to be rounded to a `long`.

Returns: the value of the argument rounded to the nearest `long` value.

See Also: [Long.MAX_VALUE](#),

Deklaration einer Methode mit Javadoc



```
01  /**
02   * Berechnet den Bruttobetrag
03   *
04   * @param Nettobetrag
05   * @param Steuersatz
06   * @return Bruttobetrag
07   */
08  public double berechneBruttobetrag(double nettobetrag, double steuersatz) {
09      //...
10  }
```

Beispiel

```
01  /**
02  * Student from HKA
03  */
04  public class Student {
05      /*
06      * Every student has a daily routine
07      * in university, which is
08      * carried out here.
09      */
10      public static void main(String[] args) {
11          boolean isStudent = true;
12      }
13  }
```

Sinnhaftigkeit von Kommentaren HKA

- Beispiel 1

```
01 // The variable „counter“ is initialized with 0
02 int counter = 0;
```

- Beispiel 2

```
01 /* since ver 3.2, library XY expects the duration in seconds,
02    not milliseconds */
03 int duration = 5 * 60 * 60;
```

- Wie sinnvoll sind diese Kommentare jeweils?

Definition: Boolescher Ausdruck



- Wahrheitswert – kann wahr oder falsch sein
- Oft das Ergebnis eines Vergleichs
- Oft als Bedingungen oder zur Entscheidungsfindung verwendet
- Mehrere Werte können verknüpft werden – zu sog. booleschen Ausdrücken
- Das Ergebnis kann in einer Variable boolean gespeichert werden

```
01 boolean isHungry = true;  
02 boolean isHome = false;
```

Boolsche Operatoren – Vergleiche + IKA

>	größer
<	Kleiner
>=	größer gleich
<=	kleiner gleich
==	gleich
!=	ungleich

Vergleiche – Beispiele

```
01 int x = 5;  
02 int y = 10;
```

- Wahr oder falsch?

1. $x < y$
2. $x > y$
3. $x == 5$
4. $x != y$
5. $x < 5$
6. $y \leq 10$

Vergleiche – Beispiele

```
01 int x = 5;  
02 int y = 10;
```

- Wahr oder falsch?

1. $x < y$ //Wahr, 5 ist kleiner als 10
2. $x > y$ //Falsch, 5 ist nicht größer als 10
3. $x == 5$ //Wahr, x ist gleich 5
4. $x != y$ //Wahr, x ist ungleich y, da 5 nicht gleich 10 ist
5. $x < 5$ //Falsch, da $x = 5$
6. $y <= 10$ //Wahr, da $y = 10$

Verknüpfungen

Verknüpfung	Sprich	Bedeutung
$a \& b$	a und b	sind a und b true, ist die Verknüpfung true
$a b$	a oder b	sind a oder b true, ist die Verknüpfung true
$A \wedge b$	entweder a oder b	ist entweder a oder b true, ist die Verknüpfung true
$!a$	nicht a	ist a true, ist die Verknüpfung mit ! false

Wahrheitstabellen

a	b	&
false	false	false
false	true	false
true	false	false
true	true	true

a	b	
false	false	false
false	true	true
true	false	true
true	true	true

a	b	$\wedge$
false	false	false
false	true	true
true	false	true
true	true	false

a	!
false	true
true	false

Gerade in der technischen Informatik wird
1 = true und **0 = false** verwendet

Beispiele



```
01 boolean getDiscount = isStudent | isUnderage
```

- Was passiert hier?
- Der Wert **getDiscount** ist wahr, wenn man Student ist **ODER** minderjährig

```
01 boolean getDiscount = (hasBirthday & isWeekend) | isSenior | hasCoupon
```

- Hier?
- Der Ausdruck ist wahr, wenn die Person Geburtstag hat **UND** es Wochenende ist **ODER** die Person Rentner ist **ODER** einen Gutschein hat

Kurzschlussoperatoren

- UND und ODER gibt es auch als sogenannte Kurzschlussoperatoren: && und ||
- Evaluierung des zweiten Operanden wird übersprungen, wenn der erste Operand false ist.
- && und || sind die bevorzugten Operatoren für boolesche Ausdrücke, weil sie Kurzschlussverhalten unterstützen und die Effizienz verbessern, indem sie unnötige Bewertungen vermeiden.

Rangfolge der Operatoren



++, --	Inkrement und Dekrement
!	Negation
*, /, %	Multiplikation, Division, Modulo
+, -	Addition und Subtraktion
==, !=, <=, >=	Vergleich
&	Und
	Oder
&&	Kurzschlussoperator Und
	Kurzschlussoperator Oder
=	Zuweisung

Unäre Operatoren

Rechenzeichen

Binäre Operatoren

Zuweisung

Beispiel



- Schreibe einen boolschen Ausdruck in Java Syntax, der einen Sportwagen erkennt

Unsere Definition eines Sportwagens:

- Maximal 2 Türen, keine Rücksitze

Außerdem:

- Höchstgeschwindigkeit von min. 200 km/h und min. Beschleunigung von 0-100 km/h in 8 Sekunden
- **oder** Höchstgeschwindigkeit von mindestens 280 km/h und mindestens 250 PS

Auszuwertende Variablen



```
01 //true, wenn das Auto Rücksitze hat
02 boolean hasBackseats;
03
04 //Anzahl Türen
05 int doors;
06
07 //Beschleunigung von 0 auf 100 in Sekunden
08 double acceleration;
09
10 //Höchstgeschwindigkeit in KM/H
11 int maxSpeed;
12
13 //Leistung in PS
14 int powerInPS;
```

Unsere Definition eines Sportwagens:

Maximal 2 Türen, keine Rücksitze

Außerdem:

Höchstgeschwindigkeit von
min. 200 km/h und min. Beschleunigung
von 0-100 km/h in 8 Sekunden

oder Höchstgeschwindigkeit von
mindestens 280 km/h und
mindestens 250 PS

Lösung

```
01 //Lösung 1
02 doors < 3
03 && !hasBackseats
04 && (( maxSpeed >= 200 && acceleration <= 8)
05 || ( maxSpeed >= 280 && powerInPS >= 250))
06
07 //Lösung 2
08 doors <= 2
09 && hasBackseats == false
10 && (( maxSpeed >= 200 && acceleration <= 8)
11 || ( maxSpeed >= 280 && powerInPS >= 250))
12
13
14
```


Vergleich von Gleitkommazahlen HKA

- Gleitkommazahlen in Java sind anfällig für Rundungsfehler
- Schon der Variablentyp kann einen Unterschied machen
- Niemals direkt (mit `==`) vergleichen!

Vergleich von Gleitkommazahlen HKA

```
01 (double)0.02 == (float)0.02f //false
```

- um sicherzugehen lieber prüfen, ob die Abweichung nur minimal ist:

```
01 Math.abs(0.02 - 0.02f) < 4.5E-10  
02 //Math.abs(zahl) berechnet den Betrag von zahl
```


- 27

Fallunterscheidung durch if-Abfragen

- Anweisung wird nur dann ausgeführt, wenn eine bestimmte Bedingung erfüllt ist:

```
01  if (Bedingung) {  
02      Anweisung1;  
03      Anweisung2;  
04      // ...  
05  } else if (andere Bedingung) {  
06      Anweisung3;  
07      // ...  
08  } else {  
09      // ...  
10  }
```

- (**else if** und **else** optional)

Beispiel



```
01  if (weHaveMate) {  
02      System.out.println("Nimm ne Mate!");  
05  } else if (weHaveCoffee) {  
06      System.out.println("Nimm nen Kaffee!");  
08  } else {  
09      System.out.println("Pech gehabt!");  
10  }
```

if-Abfragen – Syntax



- Auf die Klammer, in der die Bedingung angegeben wird, folgt kein Semikolon!
- In einem If-Block können beliebig viele Anweisungen stehen
- Die einzelnen Blöcke werden durch geschweifte Klammern getrennt

noch ein praktischeres Beispiel



```
01  int a;  
02  double b;  
03  // Variablen werden Werte zugeordnet...  
04  if (a != 0) {  
05      System.out.println("b/a: ");  
06      System.out.println(b/a);  
07  } else {  
08      System.out.println("Division durch 0!");  
09  }  
10
```

Abkürzungen bei boolschen Variablen



```
01  if (isStudent == true) {...}
02  //ist das selbe wie
03  if (isStudent) {...}
04
05  if (isStudent == false) {...}
06  //ist das selbe wie
07  if (!isStudent) {...}
```


If-Abfrage Übung



Schreibe ein Programm, das:

- Eine Variable vom Typ `int` deklariert und initialisiert
- Überprüft, ob diese Variable durch 2 teilbar ist
- Bei gerader Zahl ausgibt: "Die Zahl <Zahl> ist gerade!,"
- Bei ungerader Zahl ausgibt: "Zahl <Zahl> ist ungerade!"

If-Abfrage Übung – Lösung



```
01  int number = 5;
02  int remainder = number % 2;
03
04  if (remainder == 0) {
05      System.out.println("Die Zahl " + number + " ist gerade");
06  } else {
07      System.out.println("Die Zahl " + number + " ist ungerade");
08  }
```

Verschachtelte if-Abfragen

```
01  boolean isStudent;  
02  int age;  
03  //Werte zuweisen, berechnen...  
04  if (isStudent) {  
05      if (age < 18) {  
06          System.out.println(„zu jung!“);  
07      } else {  
08          System.out.println(„kommt rein!“);  
09      }  
10  } else {  
11      System.out.println(„kein Student“);  
12  }
```

- Vorsicht! Wird schnell unübersichtlich

Unübersichtlichkeit bei If/Else



- If/else eignet sich gut für Unterscheidungen von wahr/falsch
- Gibt es sehr viele Möglichkeiten, werden Statements schnell unübersichtlich
- Der Code wird schwieriger zu warten und zu ändern, je komplexer die Bedingungen werden.
- Erhöht das Risiko von Fehlern und unerwartetem Verhalten.

Lösung:

1. Aufteilen der Logik in kleinere, verständliche Methoden.
2. Switch-Cases

Fallunterscheidung durch Switch-Cases



- Fallunterscheidung in Abhängigkeit von **einer** Variablen
- Nur ganzzahlige Typen, **char** und **String**
- Jeder Wert darf **nur einmal** vorkommen
- Läuft durch bis **break** oder bis zum Ende des Switch-Cases
- Trifft **keiner** der beachteten Fälle ein, wird (ähnlich dem **else**) ein **default** ausgeführt, wenn vorhanden

Beispiel: Autonomes Fahren für Anfänger



```
01 String trafficLightInput = „red“;
02
03 switch (trafficLightInput) {
04     case „red“:
05         System.out.println(„Stop!“);
06         break;
07     case „green“:
08         System.out.println(„keep driving!“);
09         break;
10     case „yellow“:
11         System.out.println(„slow down“);
12         break;
13     default: //Traffic light broken
14         System.out.println(„be careful“);
15 }
```

Beispiel mit Fall-Through

```
01 String fallThrough;  
02 ...  
03 switch (fallThrough) {  
04     case 0:  
05         System.out.println(„Success!“);  
06         break;  
07     case 1:  
08         System.out.println(„Cancelled“);  
09         //Fall through  
10     case 2:  
11         System.out.println(„Failed“);  
12         break;  
13     default:  
14         System.out.println(„Unknown“);  
15 }  
16
```

Switch-Case verglichen mit if/else



```
01 char input;
02 ...
03 switch (input) {
04     case ',r' :
05         //lesen
06         break;
07     case ',q' :
08         //beenden
09         break;
10     case ',n' :
11         //Neustart
12         break;
13 }
14 }
```

```
01 ...
02 if (input == ',r') {
03     //lesen
04 } else if (input == ',q') {
05     //beenden
06 } else if (input == ',n') {
07     //Neustart
08 }
09 }
```